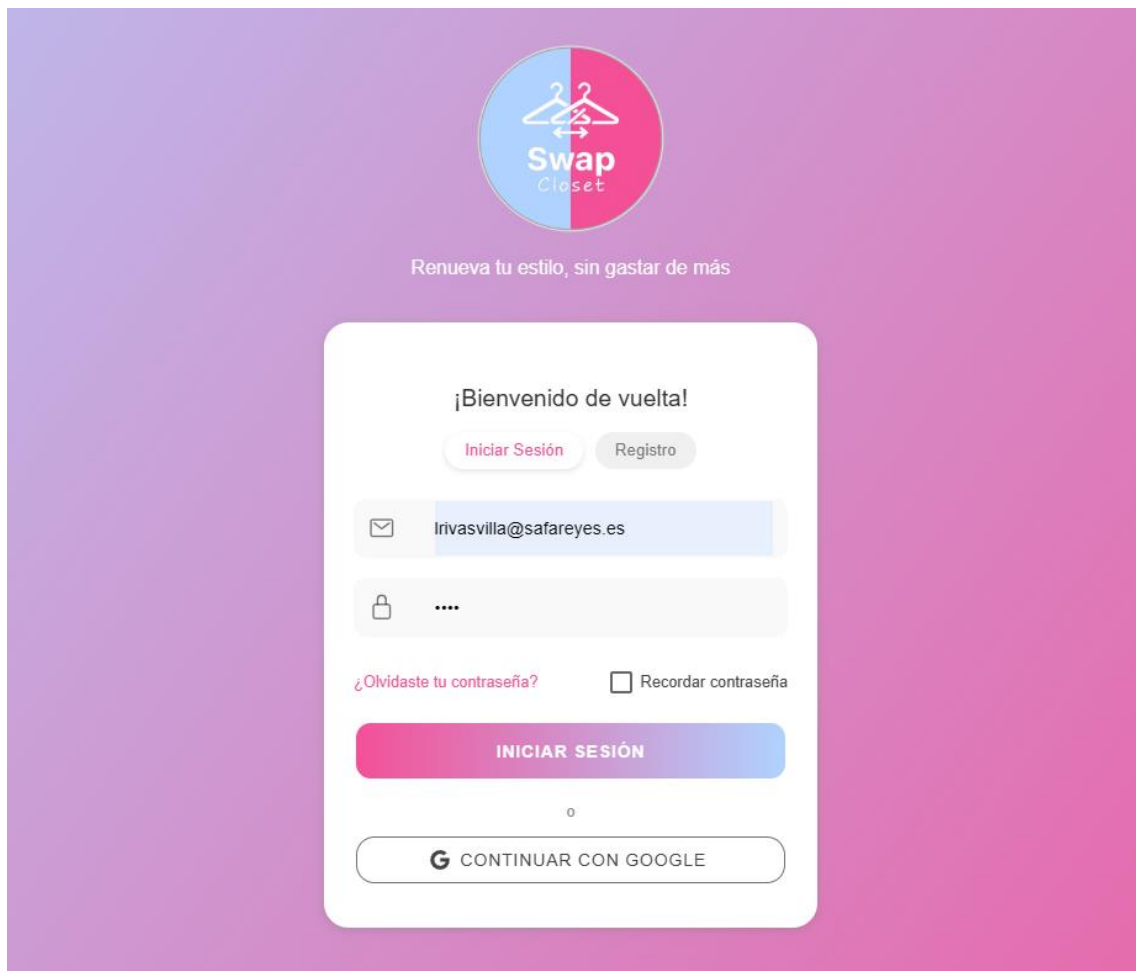


Despliegue Ionic + Angular + Spring Boot en Render y Android

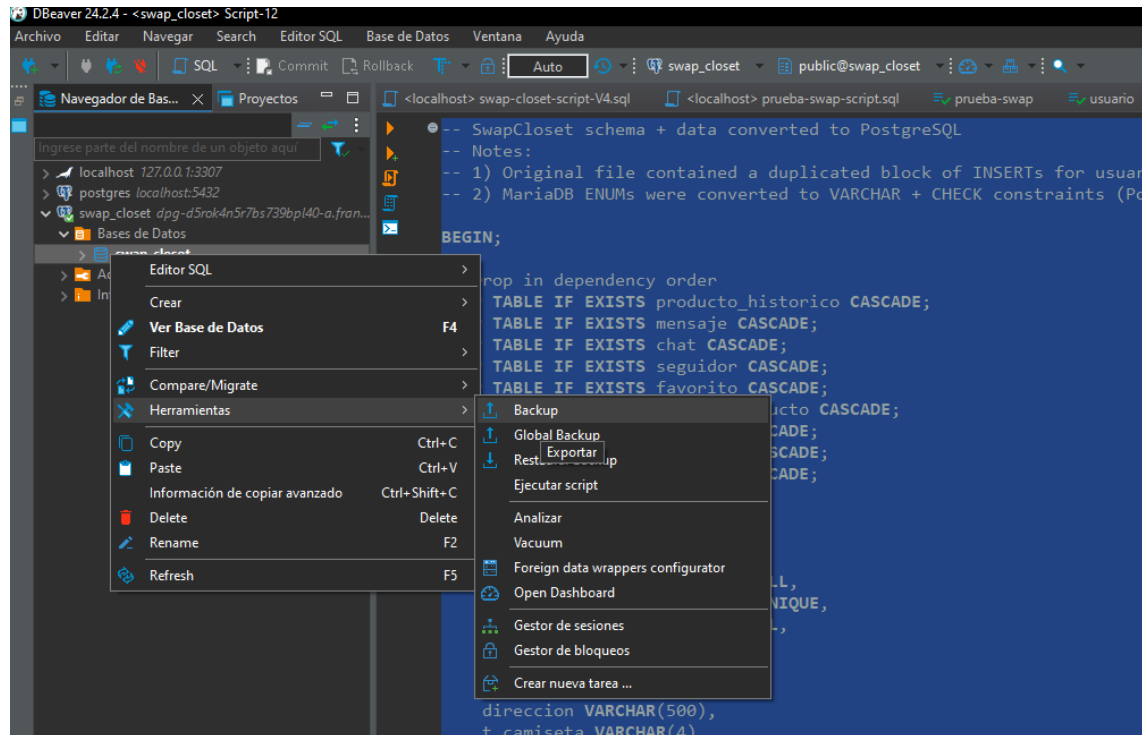


Luis Rivas Villa 2ºDAM

Despliegue en Render

1.Base de Datos

1.1 Abrir nuestra base de datos en Postgres y exportar el backup



1.2 Vamos a Render y creamos una base de datos de Postgres

New Postgres [View docs](#)

Name
A unique name for your Postgres instance.

Project Optional
Add this database to a [project](#) once it's created.
 /

Database Optional
The Postgres dbname

User Optional

Region
Your services in the same [region](#) can communicate over a [private network](#). You currently have services running in **Frankfurt**.
☒ Frankfurt (EU Central) 3 existing services
[Deploy in a new region](#) +

PostgreSQL Version

Datadog API Key Optional
The API key to use for sending metrics to Datadog. Setting this will enable Datadog monitoring.

Datadog Region Optional
The region key to use for sending metrics to Datadog.

☐ Accelerated
Memory-optimized plans offer significant performance improvements. Starting at \$160 / month plus storage.

Need a [custom instance type](#)? We support up to 1024 GB RAM, 128 CPUs, and 16 TB storage.

Storage
Your database's capacity, in GB. You can increase storage later, but you can't decrease it.
Specify 1 GB or any multiple of 5 GB.

Storage Autoscaling
When your database is 90% full, automatically increase its storage by 50% (rounded up to the nearest 5 GB).
Max one increase per 12 hours. [Learn more](#)

High Availability
Run a standby instance of your database and automatically fail over to it if the primary encounters an issue. [Learn more](#)

Monthly Total
Database instances are billed and prorated by the second.

Instance	Free	\$0 / month
Total		\$0 / month

Create Database

1.3 Comprobamos los datos para conectarnos desde dbeaver

Connections

Hostname
An internal hostname used by your Render services.

Port

Database

Username

Password

Internal Database URL

External Database URL

PSQL Command

1.4 Nos conectamos desde Dbeaver, cargamos el backup y hacemos el insert a la base de datos

General PostgreSQL Driver properties SSH SSL + Network configurations...

Server
Connect by: ☒ Host ☐ URL
URL: jdbc:postgresql://dpg-d5rok4n5r7bs739bp140-a.frankfurt-postgres.render.com:5432/swap_closet
Host: dpg-d5rok4n5r7bs739bp140-a.frankfurt-postgres.render.com Port: 5432
Database: swap_closet ☐ Show all databases

Authentication
Authentication: Database Native
Nombre de usuario: swap_closet_user
Contraseña: Save password

Advanced
Session role: Local Client: PostgreSQL 18

[Connection variables information](#) [Database documentation](#)

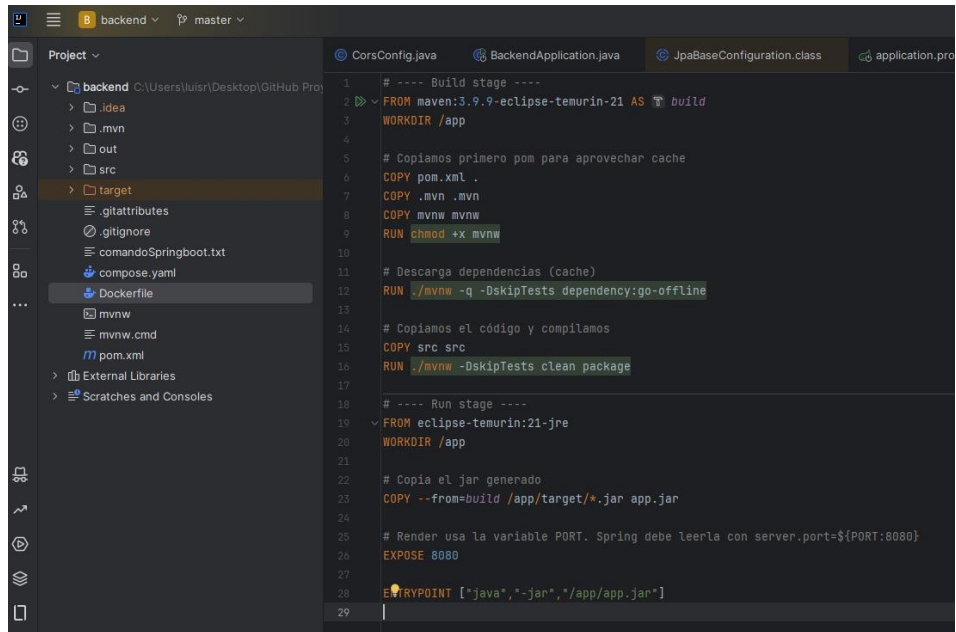
Driver name: PostgreSQL Driver Settings Licencia del driver

OK Cancel

Con eso ya tendríamos nuestra base de datos completa en Render

2.Backend

2.1 Creamos un Dockerfile

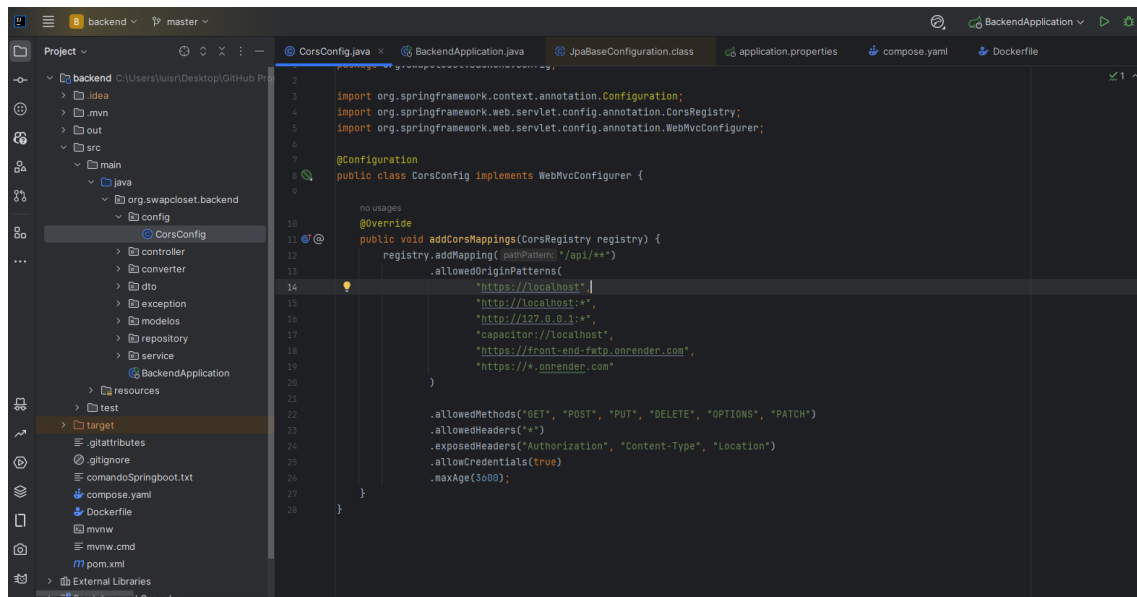


```

1  # ---- Build stage ----
2  FROM maven:3.9.9-eclipse-temurin-21 AS build
3  WORKDIR /app
4
5  # Copiamos primero pom para aprovechar cache
6  COPY pom.xml .
7  COPY .mvn .mvn
8  COPY mvnw mvnw
9  RUN chmod +x mvnw
10
11 # Descarga dependencias (cache)
12 RUN ./mvnw -q -DskipTests dependency:go-offline
13
14 # Copiamos el código y compilamos
15 COPY src src
16 RUN ./mvnw -DskipTests clean package
17
18 # ---- Run stage ----
19 FROM eclipse-temurin:21-jre
20 WORKDIR /app
21
22 # Copia el jar generado
23 COPY --from=build /app/target/*.jar app.jar
24
25 # Render usa la variable PORT. Spring debe leerla con server.port=${PORT:8080}
26 EXPOSE 8080
27
28 ENTRYPOINT ["java","-jar","/app/app.jar"]
29

```

2.2 Dejamos ajustado nuestro Cors

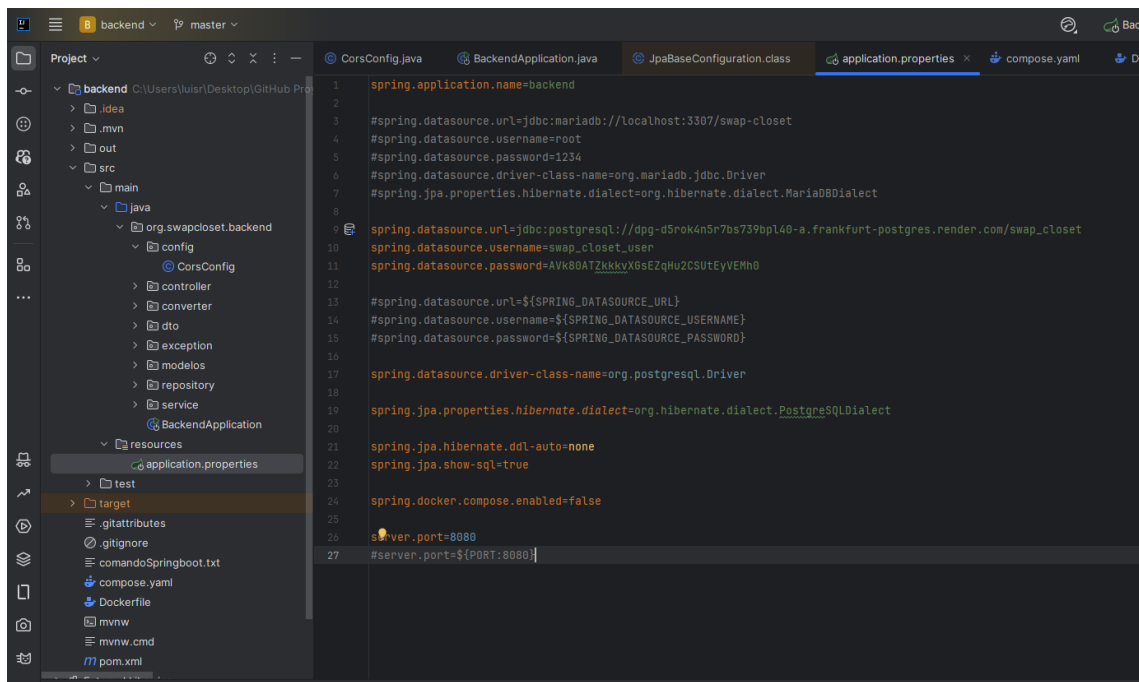


```

1  import org.springframework.context.annotation.Configuration;
2  import org.springframework.web.servlet.config.annotation.CorsRegistry;
3  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
4
5  @Configuration
6  public class CorsConfig implements WebMvcConfigurer {
7
8      no usages
9
10     @Override
11     public void addCorsMappings(CorsRegistry registry) {
12         registry.addMapping("/api/**")
13             .allowedOriginPatterns(
14                 "https://localhost" |
15                 "http://localhost:*",
16                 "http://127.0.0.1:*",
17                 "capacitor://localhost",
18                 "https://front-end-fttp.onrender.com",
19                 "https://*.onrender.com"
20             )
21
22         .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH")
23         .allowedHeaders("*")
24         .exposedHeaders("Authorization", "Content-Type", "Location")
25         .allowCredentials(true)
26         .maxAge(3600);
27     }
28 }

```

2.3 Configuramos el application.properties



The screenshot shows an IDE with the 'application.properties' file open. The file contains the following configuration:

```
1 spring.application.name=backend
2
3 #spring.datasource.url=jdbc:mariadb://localhost:3307/swap-closet
4 #spring.datasource.username=root
5 #spring.datasource.password=1234
6 #spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
7 #spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MariaDBDialect
8
9 spring.datasource.url=jdbc:postgresql://dpg-d5rok4n5r7bs739bpl40-a-frankfurt-postgres.render.com/swap_closet
10 spring.datasource.username=swap_closet_user
11 spring.datasource.password=AVk80ATZkkyX0sEZqHuZCStEyVEMh0
12
13 #spring.datasource.url=${SPRING_DATASOURCE_URL}
14 #spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
15 #spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
16
17 spring.datasource.driver-class-name=org.postgresql.Driver
18
19 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
20
21 spring.jpa.hibernate.ddl-auto=none
22 spring.jpa.show-sql=true
23
24 spring.docker.compose.enabled=false
25
26 #server.port=8080
27 #server.port=${PORT:8080}
```

En nuestro caso hemos hardcodedo para no tener que definir variables del entorno en Render.

2.4 Subimos nuestro proyecto a GitHub

Proyecto-SwapCloset-Luis-Rivas-Villa / Swap-Closet-V5-despliegue /

Irivasvilla Desplegando en Android	
Name	Last commit message
..	
backend	Desplegando en Android
frontend	login arreglado con footer
img	Subiendo la V5 con despliegue, no se toca la v4 que es donde están lo...
dump-swap_closet-202601261634.sql	Subiendo la V5 con despliegue, no se toca la v4 que es donde están lo...
swap-closet-postgres.sql	Subiendo la V5 con despliegue, no se toca la v4 que es donde están lo...
swap-closet-script-V4.sql	Subiendo la V5 con despliegue, no se toca la v4 que es donde están lo...

2.5 Creamos un Web Service en Render y le indicamos la dirección de Git de nuestro backend

WEB SERVICE

Backend Docker Free Upgrade your instance →

Connect

Service ID: srv-d5pvia4d50c73987dag

invasilla / Proyecto-SwapCloset-Luis-Rivas-Villa master

https://proyecto-swapcloset-luis-rivas-villa.onrender.com

Settings

General

Name

Backend

Edit

Region

Frankfurt (EU Central)

Instance Type

Free 0.1 CPU 512 MB

Update

Please enter your payment information to select an instance type with higher limits.

See remaining free usage, or learn about free service limits.

SwapCloset / Production Backend

Search

Build & Deploy

Repository

https://github.com/rivasvilla/Proyecto-SwapCloset-Luis-Rivas-Villa

Edit

Branch

master

Edit

Git Credentials

invasilla@safareyes.es (you)

Use My Credentials

Root Directory

Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.

Swap-Closet-V5-despliegue/backend

Edit

Build Filters

Include or ignore specific paths in your repo when determining whether to trigger an auto-deploy. Paths are relative to your repo's root directory. Learn more.

Included Paths

Changes that match these paths will trigger a new build.

+ Add Included Path

Registry Credential

No credential

Edit

Dockerfile Path

Swap-Closet-V5-despliegue/backend/ ./Dockerfile

Edit

Docker Build Context Directory

Swap-Closet-V5-despliegue/backend/ .

Edit

Docker Command

Edit

Pre-Deploy Command

Optional

Render runs this command before the start command. Useful for database migrations and static asset uploads.

Swap-Closet-V5-despliegue/backend/ \$

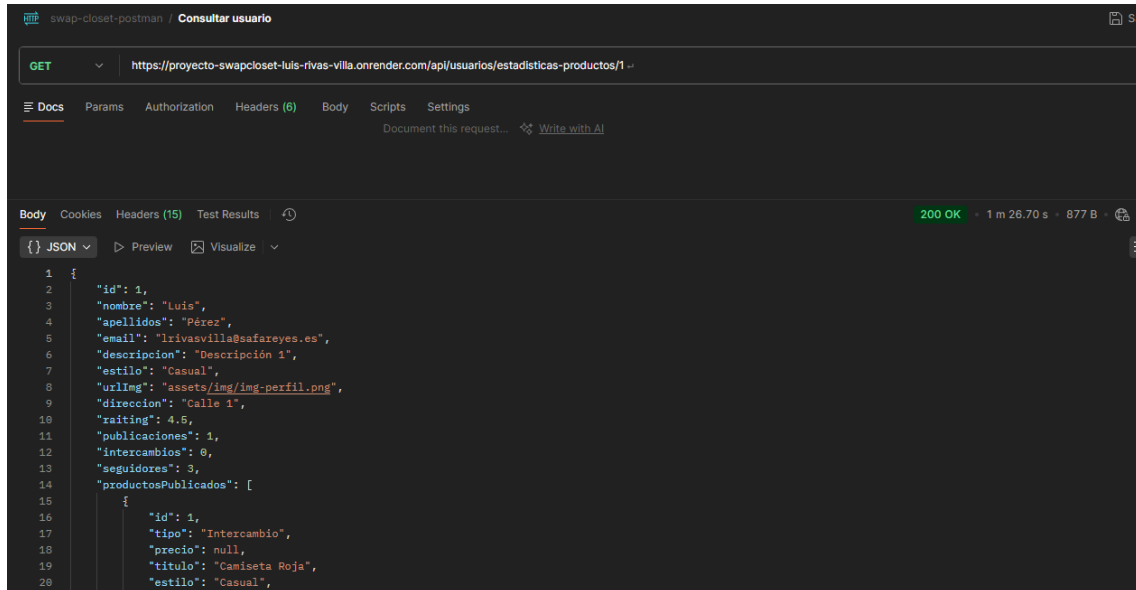
Edit

Auto-Deploy

On Commit

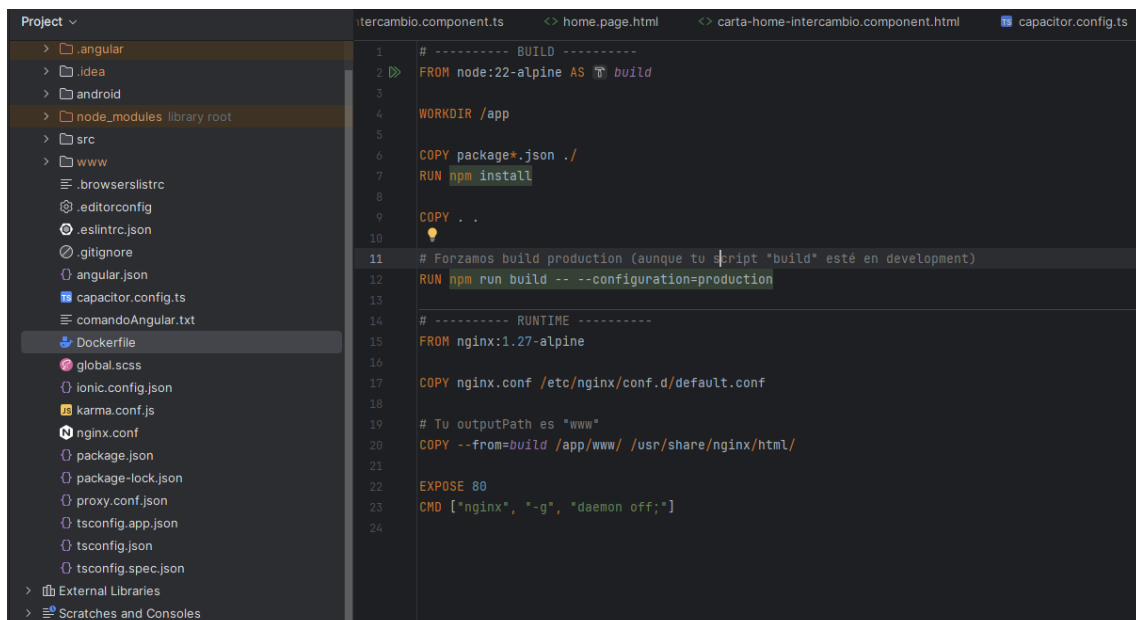
Edit

Es importante indicar la dirección de nuestro Dockerfile. Le damos a lanzar y ya tendríamos nuestro backend subido en cuanto ponga **Deployed**. Para probar qu funciona podemos hacer una petición desde postman y verificar que el backend está activo.

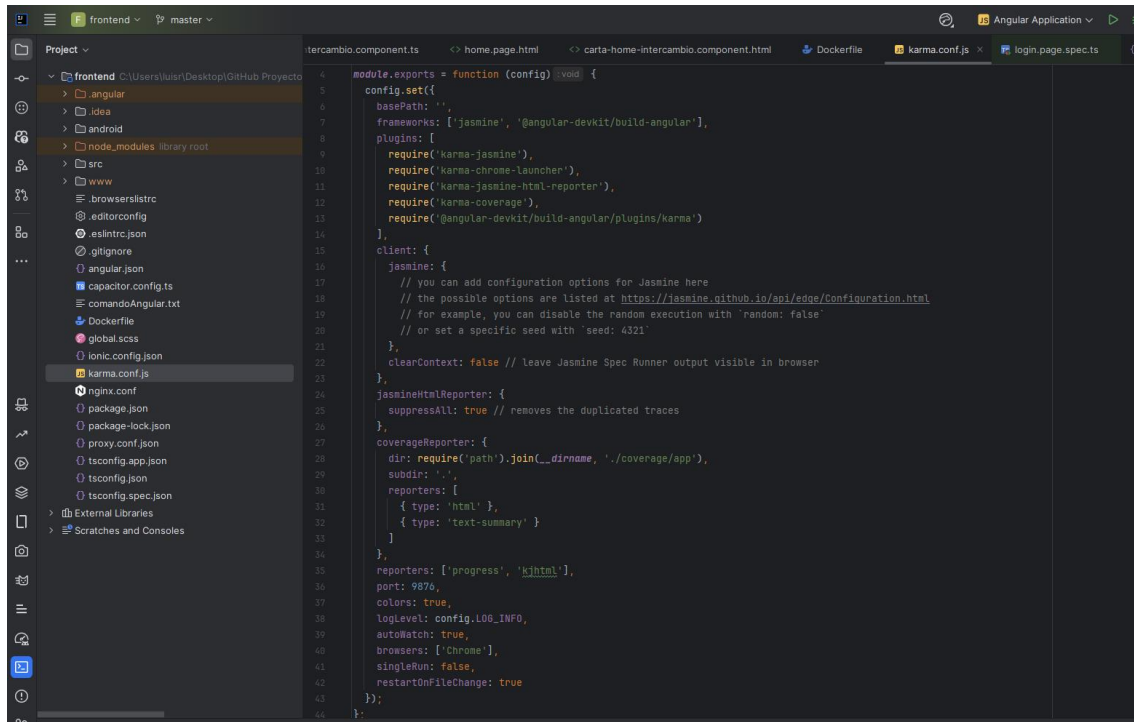


3.Front

3.1 Generamos nuestro Dockerfile



3.2 Generamos el nginx.conf



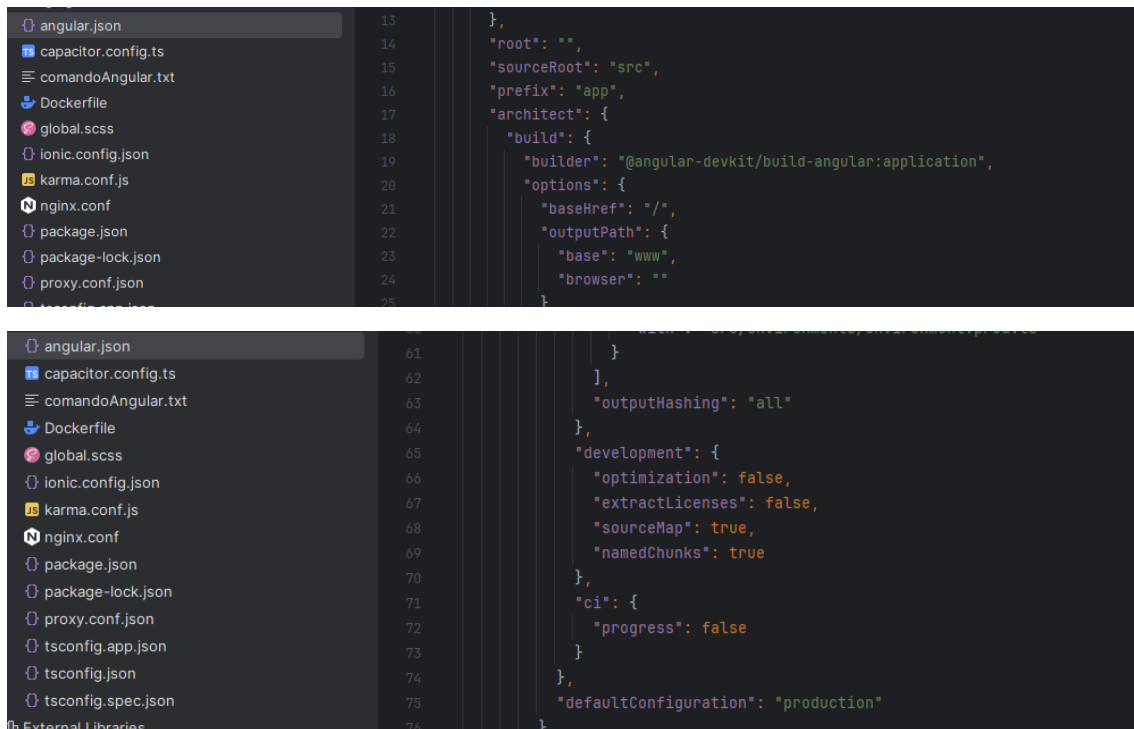
The screenshot shows the VS Code editor with the `nginx.conf` file open. The file is located in the `frontend` directory. The configuration includes the following content:

```

module.exports = function (config) {
  config.set({
    basePath: '',
    frameworks: ['jasmine', '@angular-devkit/build-angular'],
    plugins: [
      require('karma-jasmine'),
      require('karma-chrome-launcher'),
      require('karma-jasmine-html-reporter'),
      require('karma-coverage'),
      require('@angular-devkit/build-angular/plugins/karma')
    ],
    client: {
      jasmine: {
        // you can add configuration options for Jasmine here
        // the possible options are listed at https://jasmine.github.io/api/edge/Configuration.html
        // for example, you can disable the random execution with 'random: false'
        // or set a specific seed with 'seed: 4321'
      },
      clearContext: false // Leave Jasmine Spec Runner output visible in browser
    },
    jasmineHtmlReporter: {
      suppressAll: true // removes the duplicated traces
    },
    coverageReporter: {
      dir: require('path').join(__dirname, './coverage/app'),
      subdir: '.',
      reporters: [
        { type: 'html' },
        { type: 'text-summary' }
      ]
    },
    reporters: ['progress', 'kjhtml'],
    port: 9876,
    colors: true,
    logLevel: config.LOG_INFO,
    autoWatch: true,
    browsers: ['Chrome'],
    singleRun: false,
    restartOnFileChange: true
  });
};

```

3.3 Configuramos Angular.json



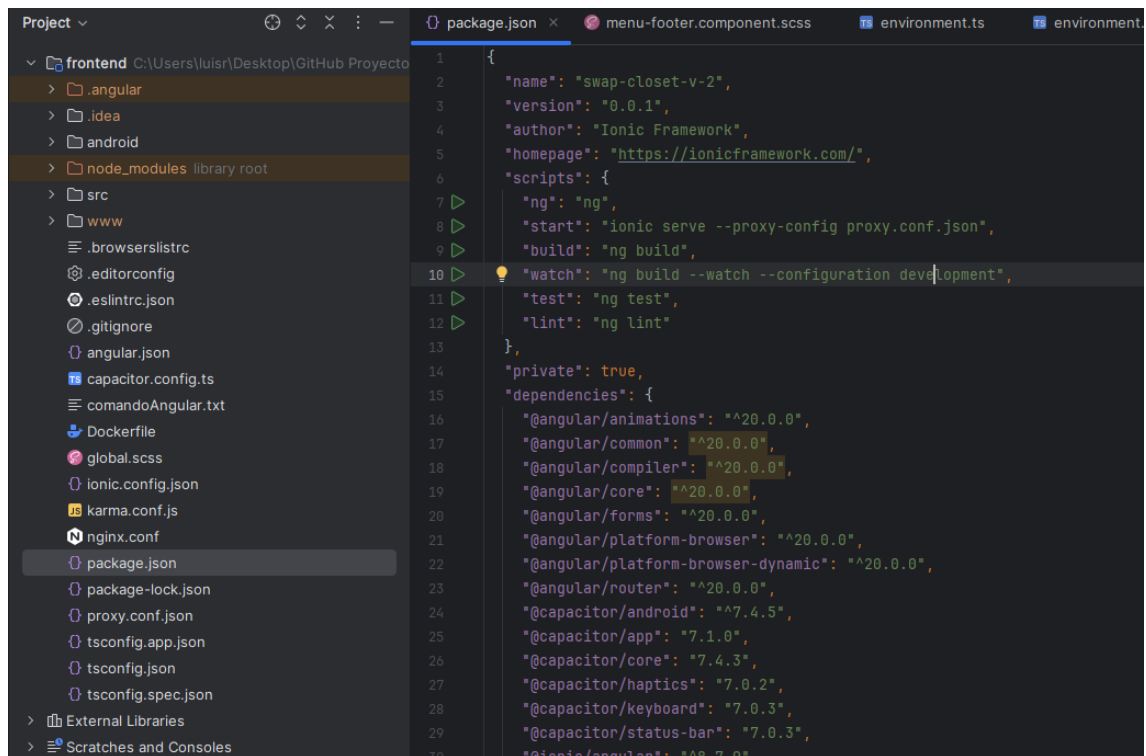
The screenshot shows the VS Code editor with the `angular.json` file open. The file is located in the `frontend` directory. The configuration includes the following content:

```

{
  "root": "",
  "sourceRoot": "src",
  "prefix": "app",
  "architect": {
    "build": {
      "builder": "@angular-devkit/build-angular:application",
      "options": {
        "baseHref": "/",
        "outputPath": {
          "base": "www",
          "browsers": ""
        }
      }
    },
    "serve": {
      "builder": "@angular-devkit/build-angular:dev-server",
      "options": {
        "port": 4200
      }
    },
    "test": {
      "builder": "@angular-devkit/build-angular:karma",
      "options": {
        "main": "src/test.ts",
        "polyfills": "src/polyfills.ts",
        "tsConfig": "src/tsconfig.spec.json",
        "karmaConfig": "karma.conf.js",
        "scripts": [],
        "styles": [
          "src/styles.css"
        ],
        "assets": [
          {
            "glob": "**/*",
            "input": "src",
            "output": "www"
          }
        ]
      }
    },
    "e2e": {
      "builder": "@angular-devkit/build-angular:protractor",
      "options": {
        "protractorConfig": "e2e/protractor.conf.js",
        "devServerConfig": {
          "port": 4200
        }
      }
    }
  }
}

```


3.4 Configuramos package.json



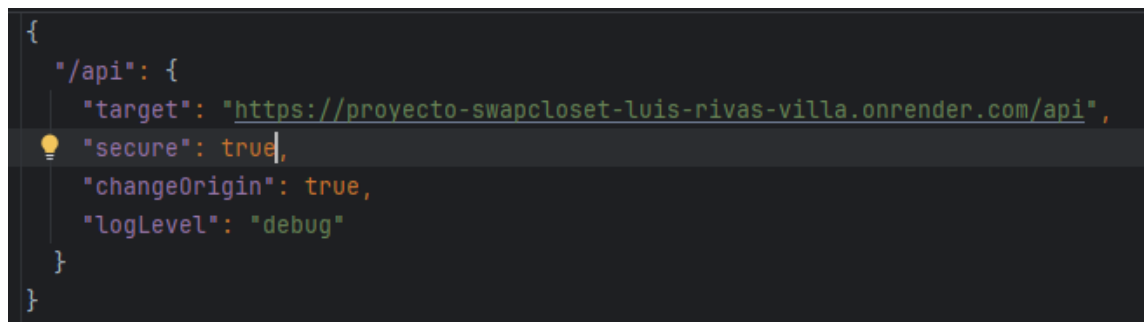
The screenshot shows an IDE with a project named 'frontend' in the left sidebar. The 'package.json' file is selected and open in the main editor. The file contains the following configuration:

```

{
  "name": "swap-closet-v-2",
  "version": "0.0.1",
  "author": "Ionic Framework",
  "homepage": "https://ionicframework.com/",
  "scripts": {
    "ng": "ng",
    "start": "ionic serve --proxy-config proxy.conf.json",
    "build": "ng build",
    "watch": "ng build --watch --configuration development",
    "test": "ng test",
    "lint": "ng lint"
  },
  "private": true,
  "dependencies": {
    "@angular/animations": "^20.0.0",
    "@angular/common": "^20.0.0",
    "@angular/compiler": "^20.0.0",
    "@angular/core": "^20.0.0",
    "@angular/forms": "^20.0.0",
    "@angular/platform-browser": "^20.0.0",
    "@angular/platform-browser-dynamic": "^20.0.0",
    "@angular/router": "^20.0.0",
    "@capacitor/android": "^7.4.5",
    "@capacitor/app": "7.1.0",
    "@capacitor/core": "7.4.3",
    "@capacitor/haptics": "7.0.2",
    "@capacitor/keyboard": "7.0.3",
    "@capacitor/status-bar": "7.0.3",
    "@ionic/angular": "^8.7.0"
  }
}

```

3.5 El proxy.conf.json



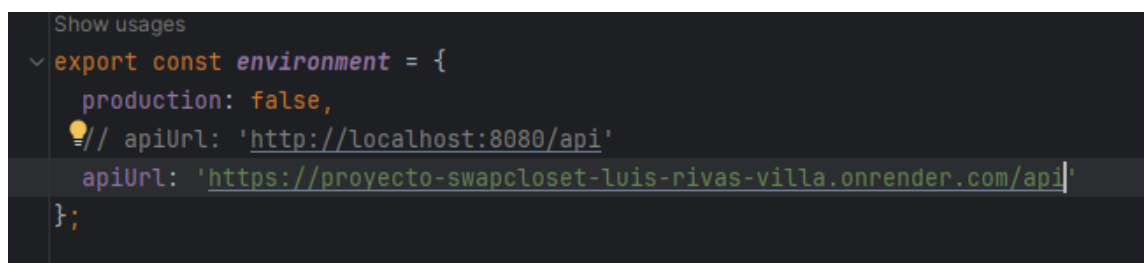
The screenshot shows the 'proxy.conf.json' file in an IDE. The file contains the following configuration:

```

{
  "/api": {
    "target": "https://proyecto-swapcloset-luis-rivas-villa.onrender.com/api",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  }
}

```

3.6 El enviroment.ts



The screenshot shows the 'environment.ts' file in an IDE. The file contains the following configuration:

```

export const environment = {
  production: false,
  apiUrl: 'https://proyecto-swapcloset-luis-rivas-villa.onrender.com/api'
};

```

3.7 El enviroment.prod.ts

```
no usages
export const environment = {
  production: true,
  apiUrl: 'https://proyecto-swapcloset-luis-rivas-villa.onrender.com/api'
};
```

3.8 Tenemos que tener en cuenta que los servicios deben ser:

```
@Injectable({
  providedIn: 'root'
})
export class UsuarioService {

  private http :HttpClient = inject(HttpClient);
  private readonly apiUrl :string = environment.apiUrl + '/usuarios';

  Show usages
  getUsuario(id: number): Observable<UsuarioDTO> {
    return this.http.get<UsuarioDTO>(`${this.apiUrl}/${id}`)
  }
}
```

Y además, **NO** debemos usar **ionicModule** en nuestro proyecto, debemos importar los componentes que necesitemos, ya que, al compilar, no cargarán los componentes. También podemos probar si nuestro front se conecta al back que ya tenemos en Render.

3.9 Crear Web Service para el Front

The screenshot shows the Render web service interface. At the top, there are tabs for 'WEB SERVICE', 'Front-end', 'Docker', 'Free', and 'Upgrade your instance ->'. Below these, the service ID is 'srv-d5saud3uibrs739oh7ag'. The service is named 'Front-end' and is located in 'Frankfurt (EU Central)'. The instance type is 'Free' with '0.1 CPU' and '512 MB'. There is a message: 'Please enter your payment information to select an instance type with higher limits.' and a link to 'See remaining free usage, or learn about free service limits.'

Build & Deploy**Repository**

The repository used for your Web Service.

https://github.com/lrivasvilla/Proyecto-SwapCloset-Luis-Rivas-Villa

[Edit](#)

Branch

The Git branch to build and deploy.

master

[Edit](#)

Git Credentials

User providing the credentials to pull the repository.

lrivasvilla@safareyes.es (you)

[Use My Credentials](#)

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Swap-Closet-V5-despliegue/frontend

[Edit](#)

Build Filters

Include or ignore specific paths in your repo when determining whether to trigger an auto-deploy. Paths are relative to your repo's root directory. [Learn more](#).

Included Paths

Changes that match these paths will trigger a new build.

[Edit](#)

Registry Credential

If your service pulls private Docker images from a registry, specify a credential that can access those images. Manage your credentials in [Settings](#).

No credential

[Edit](#)

Dockerfile Path

The path to your service's Dockerfile, relative to the repo root. Defaults to `./Dockerfile`.

Swap-Closet-V5-despliegue/frontend/ ./Dockerfile

[Edit](#)

Docker Build Context Directory

The path to your service's [Docker build context](#), relative to the repo root. Defaults to the root.

Swap-Closet-V5-despliegue/frontend/ .

[Edit](#)

Docker Command

Optionally override your Dockerfile's `CMD` and `ENTRYPOINT` instructions with a different command to start your service.

[Edit](#)

Pre-Deploy Command Optional

Render runs this command before the start command. Useful for database migrations and static asset uploads.

Swap-Closet-V5-despliegue/frontend/ \$

[Edit](#)

Auto-Deploy

By default, Render automatically deploys your service whenever you update its code or configuration.

On Commit

[Edit](#)

Con eso ya deberíamos de tener nuestro front lanzado

Despliegue en Android

0. Punto de partida

- Proyecto **Ionic + Angular** funcionando en web
- Backend **Spring Boot** desplegado (Render)
- Node.js, npm, Ionic CLI instalados
- Android Studio instalado

1. Abrir el frontend en Android Studio

1. Abrir **Android Studio**
2. Seleccionar **Open**
3. Elegir la carpeta:
4. frontend/

2. Instalar dependencias del proyecto

En la terminal integrada de Android Studio (ruta frontend/):

npm install

- Esperar a que termine sin errores

3. Instalar Capacitor Android (versión compatible)

Instalar Capacitor Android y CLI **misma major que core**:

npm install @capacitor/android@7 @capacitor/cli@7

4. Generar el build web (Angular / Ionic)

ionic build

Esto genera la carpeta **www/** con la app web compilada.

5. Añadir plataforma Android

npx cap add android

Resultado esperado:

- Se crea la carpeta android/
- Se copian los assets web

Despliegue

- Se sincronizan plugins

6. Sincronizar Android (cada cambio de frontend)

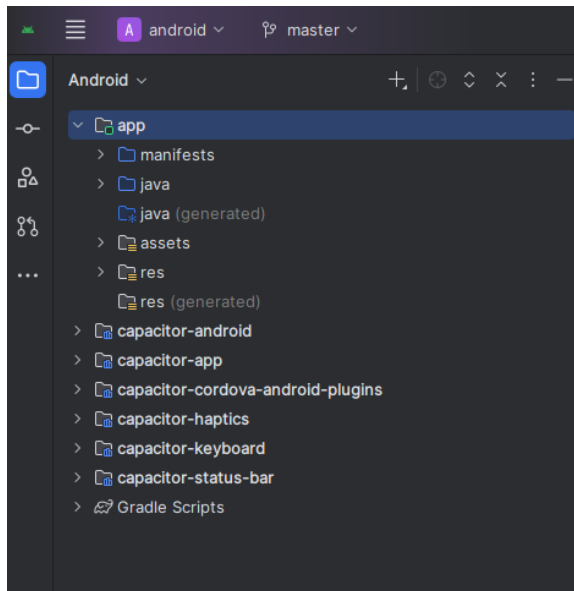
`npx cap sync android`

- Copia el build web a Android
- Actualiza plugins

7. Abrir el proyecto Android nativo

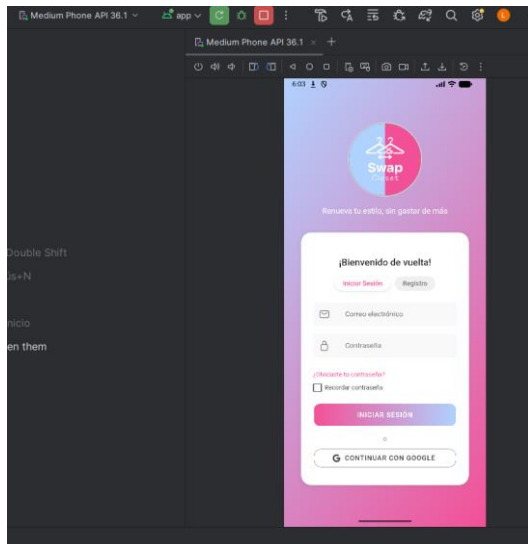
`npx cap open android`

Esto abre **Android Studio** con el proyecto Android real.



8. Ejecutar la app en emulador o dispositivo

1. En Android Studio, arriba a la derecha
2. Seleccionar un emulador o móvil físico
3. Pulsamos **Run**
4. Resultado esperado:
 - La app arranca
 - Se navega correctamente



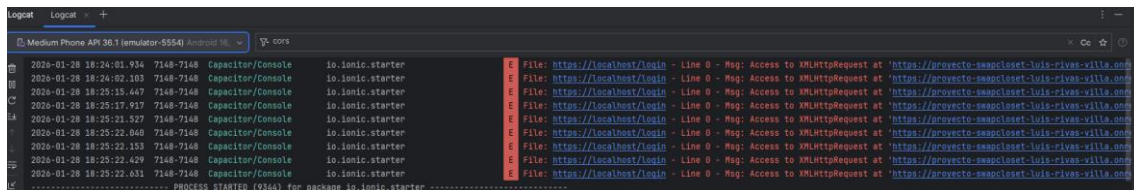
9. Detectar y resolver error de CORS

En Logcat aparece:

Access to XMLHttpRequest ... blocked by CORS policy

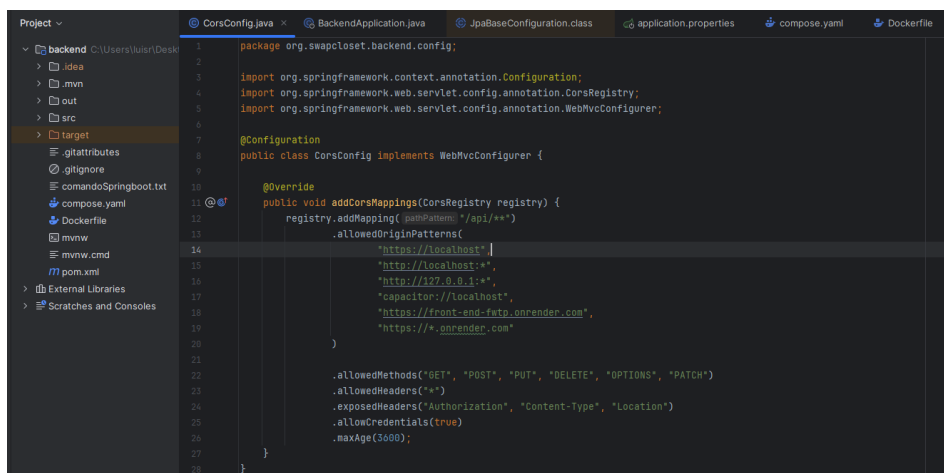
Origin: https://localhost

Esto ocurre porque Capacitor usa **https://localhost** como origen.



10. Configurar CORS en Spring Boot

- Redeploy del backend
- Probar login de nuevo



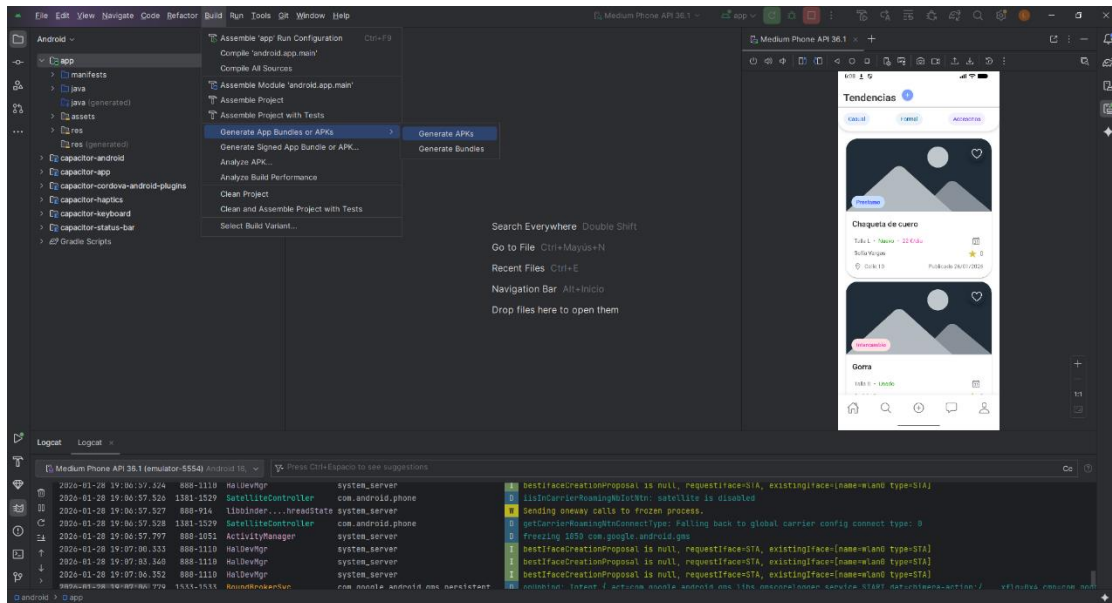
11. Generar APK o AAB (exportar la app)

En Android Studio:

Build → Generate Signed Bundle / APK

Elegir:

- **APK** → pruebas
- **Android App Bundle (AAB)** → Google Play



12. Resultado final

- App Android funcionando
- Backend conectado
- Lista para instalar o subir a Google Play

Nota: Cada vez que cambiemos el frontend:

ionic build

npx cap sync android